5. RNN

Q.1 Describe the sequence learning problem.

⇒

- In fully connected neural network (FCNN) and Convolutional Neural Network (CNN):

i) the output at any time step is independent of the previous layer input/output.

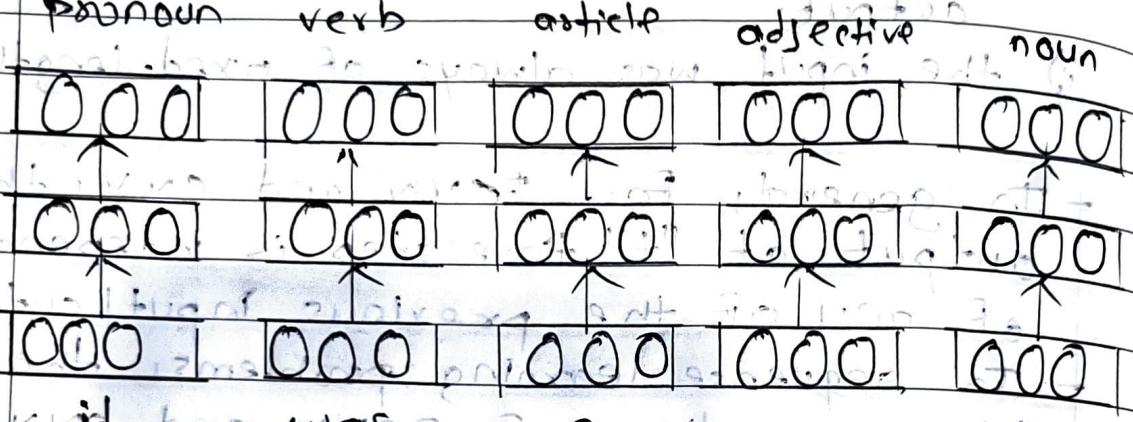
ii) the input was always of fixed-length

- In general, for FCNN and CNN, the output at "t" time step is independent of any of the previous input/output.
- In sequence learning problems, the "two properties of FCNN and CNN do not hold" and the output at any time step dependent on previous input/output and length of the input is not fixed.
- Some examples of Sequence learning are:

Ex.1: Part of speech tagging.

- Given a sequence of words, the idea is to predict part of speech tag for each word.
- Whether the word is noun, pronoun, verb, article and adjective and so on.
- Here the output depends not only on the current input but also on previous input.

- For example, the current input as "movie" and the previous input was "awesome" which is an adjective, the moment model sees an adjective, it would be more or less confident that the next word is actually going to be a noun.



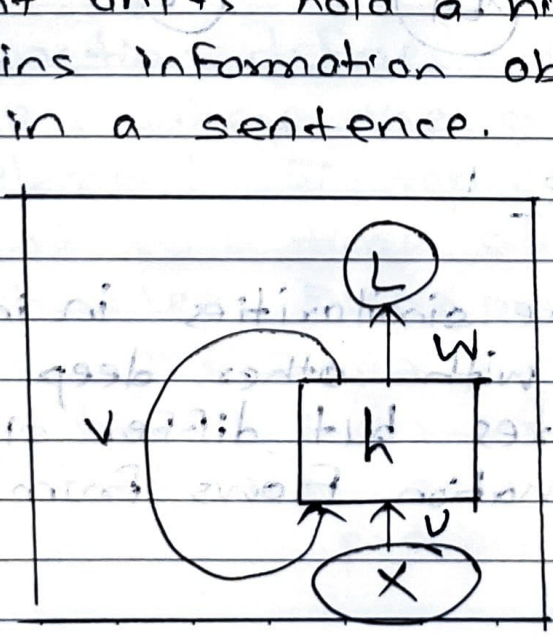
- So the confidence in predicting that the movie (word) is a noun would be higher if the previous input was an adjective.
- There is this dependency where the current output depends not only on the current input but not on the previous input as well.

Q.2 Discuss RNN in detail

=>
- RNNs are called recurrent because they perform the same task for every element of a sequence, with the output being dependent on the previous computations.
- RNN allows the network to remember past information by feeding the output from one step into next step.
- This helps the network understand the context of what has already happened and make better predictions based on that.
- For example, when predicting a next word in a sentence the RNN uses the previous words to help decide what word is most likely to come next.
- Key Components of RNNs:

1) Recurrent Neurons

=> The fundamental processing unit in RNN is a Recurrent Unit.
- Recurrent units hold a hidden state that maintains information about previous inputs in a sentence.

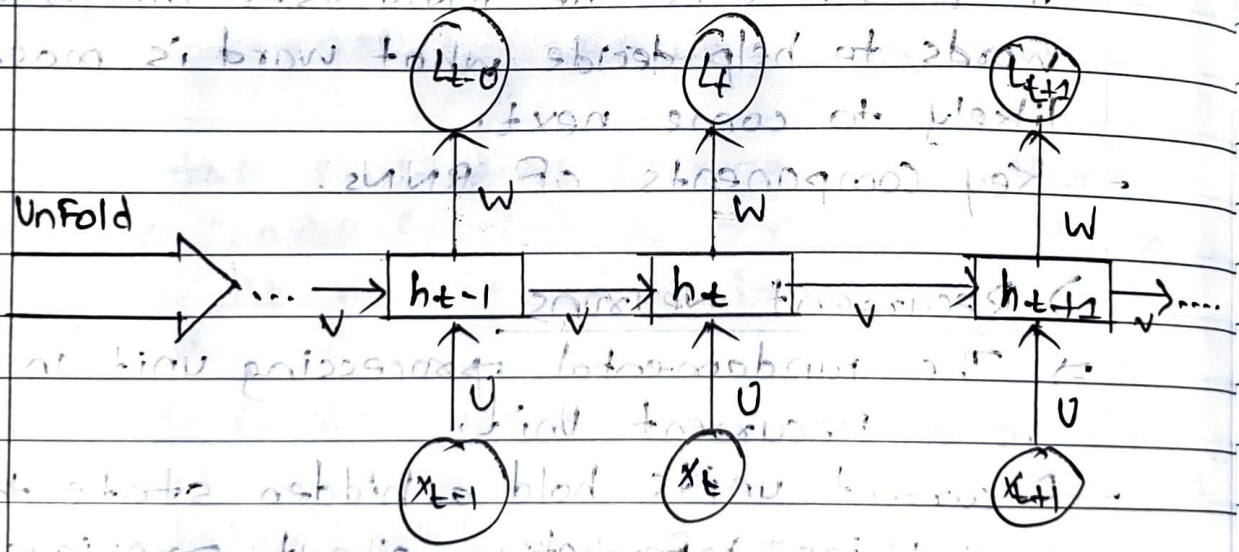


- Recurrent ~~nets~~ units can remember information from prior steps by ~~forwarding~~ feeding back their hidden state, allowing them to capture dependencies across time.

2) RNN Unfolding

⇒ RNN unfolding is a process of expanding the recurrent structure over time steps.

- During unfolding each step of the sequence is represented as a separate layer in a series illustrating how information flows across each time step.



- Architecture :

- RNNs share similarities in input and output structures with other deep learning architectures but differ significantly in how information flows from input to output.

- Unlike traditional deep neural networks, where each dense layer has distinct weight matrices, RNN uses shared weights across time steps, allowing them to remember information over sequences.
- In RNNs, the hidden state Hi is calculated for every input Xi to retain sequential dependencies.
- The computation follow these core formulas:

1) Hidden state calculation:

$$h = \sigma(U \cdot X + W \cdot h_{t-1} + B)$$

2) Output Calculations:

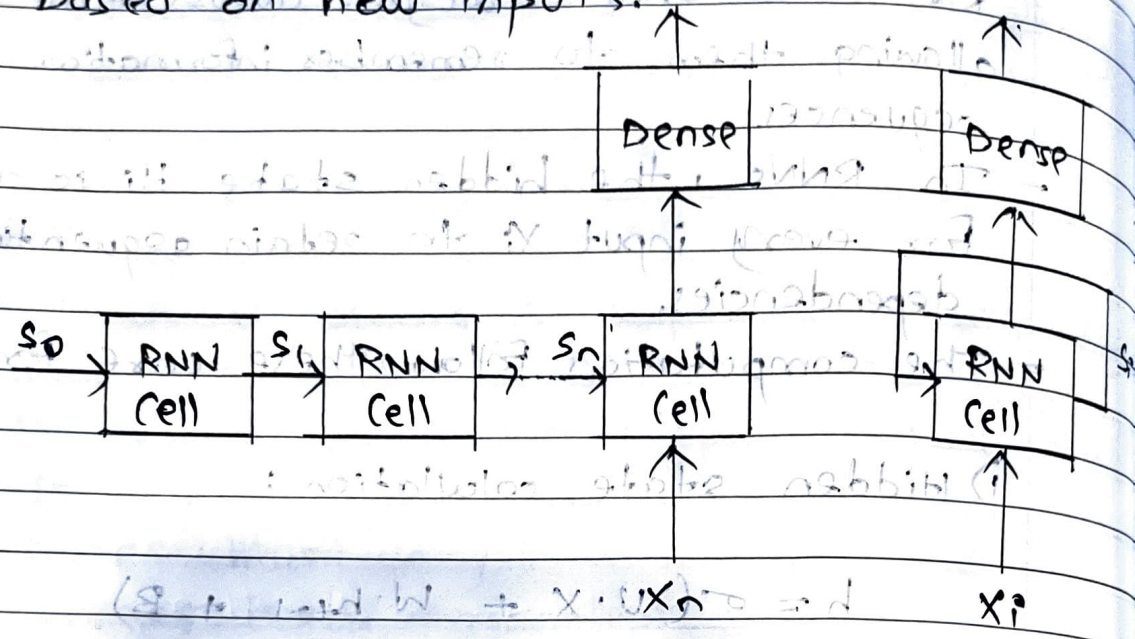
$$Y = O(V \cdot h + C)$$

3) Overall function:

$$Y = F(X, h, W, U, V, B, C)$$

- This function defines the entire RNN operation, where the state matrix S holds each element Si representing the network's state at each time step i.
- At each step RNNs process units with a fixed activation function.
- These units have an internal hidden state that acts as memory that retains information from previous time step.

- This allows memory allows the network to store past knowledge and adapt based on new inputs.



- Updating Hidden states in RNNs:

1) State update:

$$h_t = F(h_{t-1}, x_t)$$

2) Activation Function:

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t)$$

3) Output Calculations:

$$y_t = W_{hy} \cdot h_t$$

Q.3 Explain the significance of vanishing and exploding gradients.

⇒
- Two common problems that occur during the backpropagation of time-series data are the vanishing and exploding gradients.

1) Vanishing Gradient

⇒ The vanishing gradient problem is essentially a situation in which a deep multilayer Feed-forward network or a RNN does not have the ability to propagate useful gradient information from the output end of model back to the layers near the input end of the model.
- Vanishing gradient does not necessarily imply that the gradient vector is all zero.
- It implies that the gradients are minuscule, which could cause the learning to be very slow.
- Following are some indications of vanishing gradient problem:

a) The parameters of the higher layers change to a great extent, while the parameters of lower layers barely change.
b) The model weights could become 0 during training.
c) The model learns at a particularly slow pace and the training could stangate

at a very early phase after only a few iterations.

- Methods proposed to overcome the vanishing gradient problem are:

1) Batch Normalization

⇒ Batch normalization normalizes the inputs of each layer reducing internal covariate shift.

2) ReLU

⇒ Activation function like ReLU can be used. With ReLU, the gradient is 0 for negative and zero input and it is 1 for positive input, which helps to alleviate the vanishing gradient issue.

3) ResNets

⇒ Skip connections, such as ResNets, allow the gradient to bypass certain layers during backpropagation.

- This facilitates the flow of information through the network, preventing gradients from vanishing.

4) LSTM

⇒ LSTM is designed to address the vanishing gradient problem in sequences by incorporating gating mechanisms.

ii) Exploding Gradient

⇒ Exploding gradients are a problem when large error gradients accumulate and result in very large updates to neural network model weights during training.

- A gradient calculates the direction and magnitude during the training of neural network and it is used to teach network weights in the right direction by the right amount.
- When there is an error gradient, the explosion of components may grow exponentially.
- When large error gradients accumulate, the model may become unstable and unable to learn from training data.
- At an extreme, the value of weights can become so large as to overflow and result in NaN values.
- Some indications to determine whether model is suffering from exploding gradients:

a) The model does not learn much on training data therefore resulting in poor loss.
b) The model is unstable, resulting in large changes in loss for each update.
c) After sometime, the model loss goes to NaN during training.

- Solution to overcome this problem!

a) Gradient Clipping.

⇒ It sets a maximum threshold for the magnitude of gradients during backpropagation.

- Any gradients exceeding the threshold is clipped to the threshold value, preventing it from growing unbounded.

b) Batch Normalization

⇒ This technique normalizes the activations within each mini-batch, effectively scaling the gradients and reducing their variance.

c) Regularization.

d) LSTM network.

Q.4 Difference between Feed-forward neural network and RNN

=>

Feed-forward Neural Network	RNN
1). Data flows in one direction - from input to output, without loops.	1) Data flows through recurrent connections, where output from previous steps is fed back into the network.
2) Has no memory of past inputs; each input is processed independently.	2) Maintains a memory of previous inputs using hidden states, making it ideal for sequential data.
3) Works well for static input, like images classification or tabular data	3) Designed for sequential data, such as text, time-series, speech.
4) Different weights for each layer; no parameter sharing.	4) Shares parameters across time steps, making it more efficient for sequences.
5) Simpler and faster to train.	5) More complex due to BPTT.

Q.5 Explain LSTM architecture.

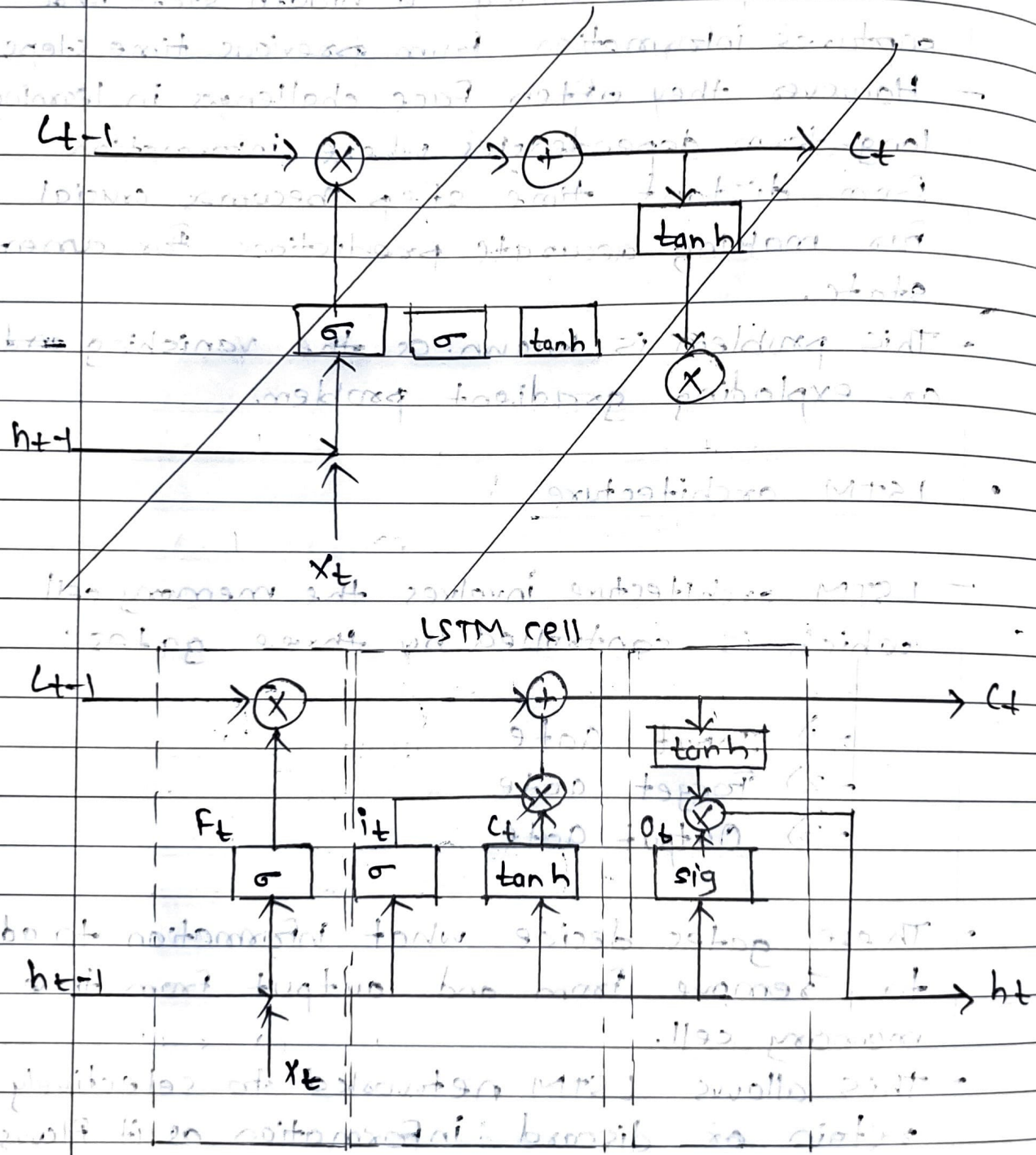
=>

- Long Short-Term memory (LSTM) is an enhanced version of RNN.
- RNNs are designed to handle sequential data by maintaining a hidden state that captures information from previous time steps.
- However they often face challenges in learning long-term dependencies where information from distant time step becomes crucial for making accurate predictions for current state.
- This problem is known as the vanishing or exploding gradient problem.

- **LSTM architecture :**

- LSTM architecture involves the memory cell which is controlled by three gates:
  - 1) Input Gate
  - 2) Forget gate
  - 3) Output Gate.
- These gates decide what information to add to, remove from and output from the memory cell.
- This allows LSTM networks to selectively retain or discard information as it flows through the network which allows them to learn long-term dependencies.

- The network has a hidden state which is like its short-term memory.
- This memory is updated using the current input, the previous hidden state and the current state of the memory cell.



Forget Gate : Input Gate : Output gate

1) Forget gate

⇒ The information that is no longer useful in the cell state is removed with forget gate.

- Two inputs x_t and h_{t-1} are fed to the gate and multiplied with weight matrices followed by the addition of bias.
- The resultant is passed through an activation function which gives a binary output.
- If for a particular cell state the output is 0, the piece of information is forgotten and for output 1, the information is retained for future use.
- The equation for forget is :-

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_F)$$

where,

W_F = weight matrix,

$[h_{t-1}, x_t]$ = concatenation of current input & previous hidden state,

b_F = bias,

σ = sigmoid activation function

2) Input gate

⇒ The addition of useful information to the cell state is done by the input gate.

- First, the information is regulated using the sigmoid function and filter the values to be remembered similar to the forget gate using inputs h_{t-1} and x_t.

- Then, a vector is created using a tanh function that gives an output from -1 to 1, which contains all possible values from h_{t-1} and x_t.
- The equation of input gate is:

$$\hat{i}_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\hat{C}_t = \tanh(W_c [h_{t-1}, x_t] + b_c)$$

$$C_t = f_t \odot C_{t-1} + \hat{i}_t \odot \hat{C}_t$$

Where,

$\odot$ denotes element-wise multiplication
tanh = activation function.

3) Output gate

⇒ The task of extracting useful information from current cell state to be presented as output is done by output gate.

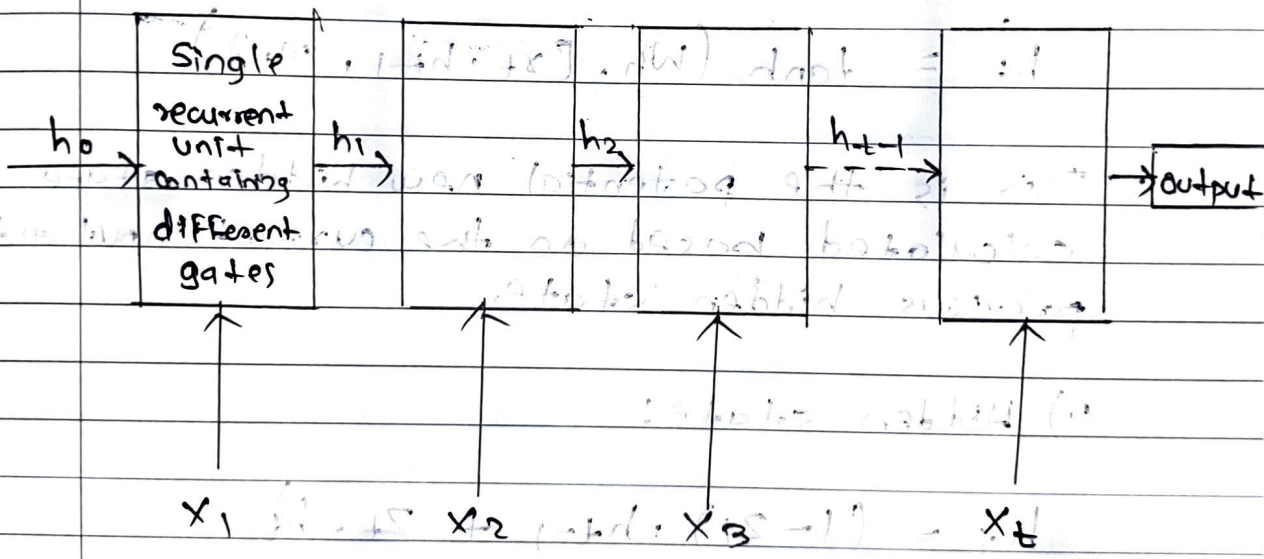
- First, a vector is generated by applying tanh function on the cell.
- Then, the information is regulated using the sigmoid function and filter by the values to be remembered using inputs h_{t-1} and x_t.
- The equation of output gate is:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Q.6 Explain the working of Gated Recurrent Unit. (GRU)

⇒

- The core idea behind GRUs is to use gating mechanisms to selectively update the hidden state at each time step allowing them to remember important information while discarding irrelevant details.
- GRUs aims to simplify the LSTM architecture by merging some of its components and focusing on two main gates:
  - 1) Update Gate
  - 2) Reset Gate



1) Update Gate (z_t)

⇒ This gate decides how much information from previous hidden state should be retained for the next time step.

2) Reset Gate

⇒ This gate determines how much of the past hidden state should be forgotten.

- Equations for GRU Operations.

1) Reset Gate:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

2) Update Gate:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

3) Candidate hidden state:

$$h'_t = \tanh(W_h \cdot [r_t \cdot h_{t-1}, x_t])$$

- This is the potential new hidden state calculated based on the current input and previous hidden state.

4) Hidden state:

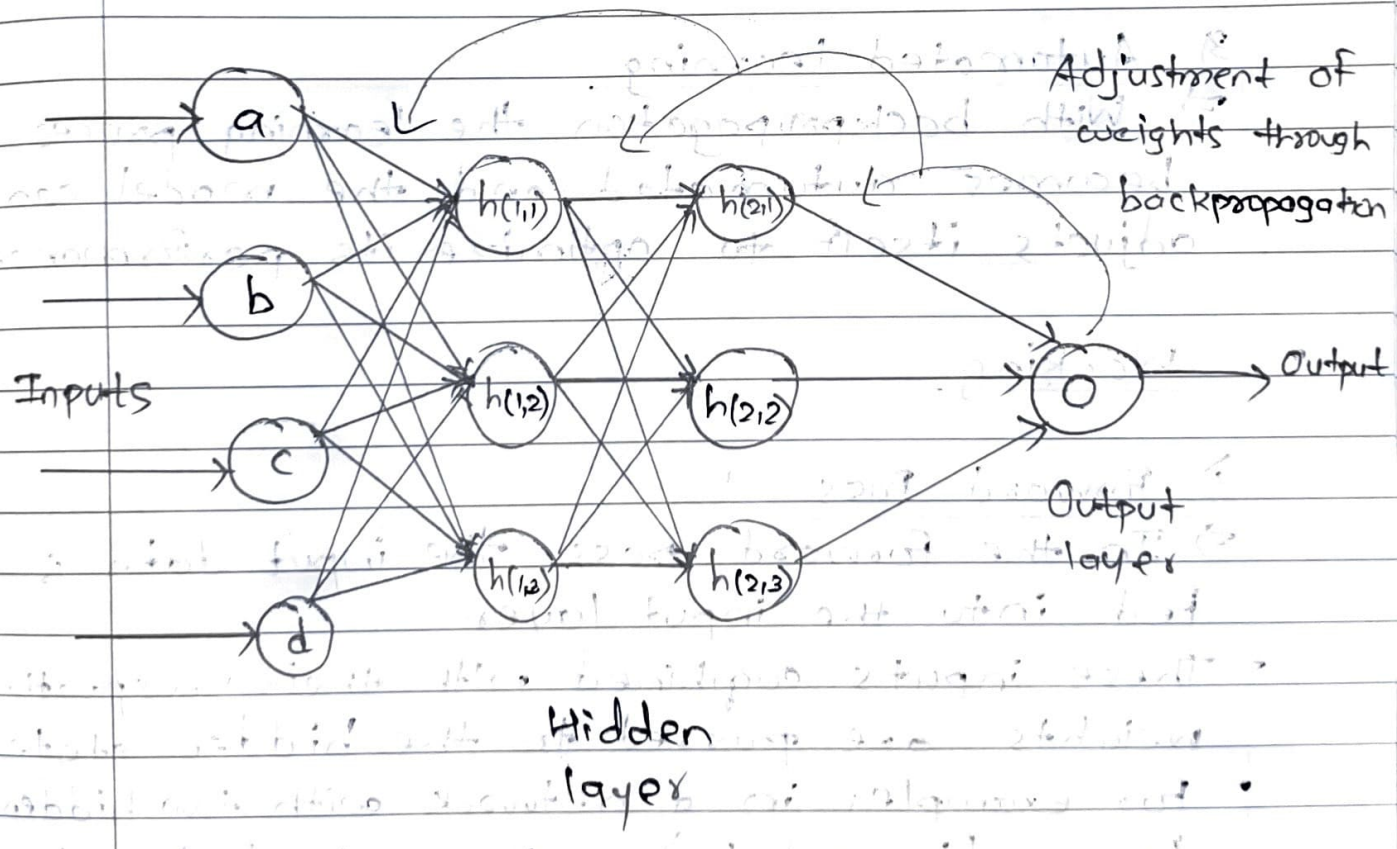
$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot h'_t$$

- The final hidden state is weighted average of previous hidden state h_{t-1} and the candidate hidden state h'_t based on the update gate z_t.

Q. 7 Describe the backpropagation training algorithm:

⇒

- Backpropagation is a technique used in deep learning to train artificial neural network Feed-forward network.
- It works iteratively to adjust weights and bias to minimize the cost function.
- Backpropagation uses optimization algorithms like gradient descent or stochastic gradient descent.
- Following figure shows the illustration of how the backpropagation works by adjustment of weights:



- Backpropagation plays a critical role in how neural networks improve over time:

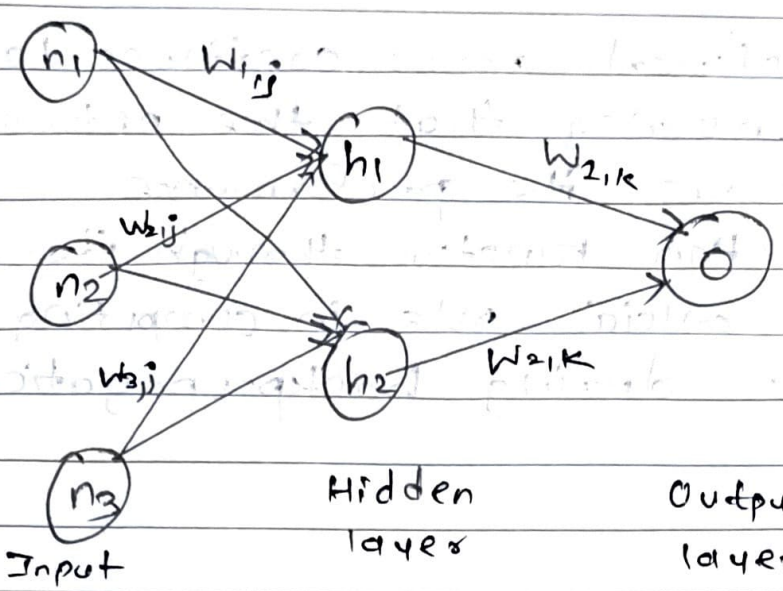
1) Efficient weight update
⇒ It computes the gradient of the loss function with respect to each weight using the chain rule making it possible to update weights efficiently.

2) Scalability
⇒ the backpropagation algorithm scales well to networks with multiple layers and complex architectures making deep learning feasible.

3) Automated learning
⇒ With backpropagation the learning process becomes automated and the model can adjusts itself to optimize its performance.

- Working

1) Forward Pass :
⇒ In the forward pass, the input data is fed into the input layer.
- These inputs combined with their respective weights are passed to the hidden state.
- For example, in a network with two hidden layers (h1 and h2), the output from h1 serves input to h2.



- Each hidden layer computes the weighted sum of the inputs then applies an activation function like ReLU to obtain output (O).
- The output is passed to the next layer where an activation function such as softmax converts the weighted outputs into probabilities for classification

2) Backward Pass

⇒ In the backward pass the error is propagated back through the network to adjust the weights and biases.

- One common method for error calculation is MSE:

$$MSE = (\text{Predicted Output} - \text{Actual output})^2$$

- Once the error is calculated the network adjusts weights using gradient which are computed with the chain rule.
- These gradient indicates how much each weight and bias should be adjusted to minimize the error in the next iteration.

- The backward pass continues layer by layer ensuring that the network learns and improves its performance.
- the activation function through its derivative plays a crucial role in computing these gradients during backpropagation.